Name: Abhay vaishnav

Subject: Major 1-OOPs Practical Assignment

Course: B.Sc.(CS) IV Semester

Roll No.: 24070102

Department: CSIT

Guided By:

Prof. Ratnesh Srivastava

1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

Ans: **PROGRAM:**

```java
import java.io.*;
import java.util.*;

// Farmer class holds data of one farmer
class Farmer {
    String name, fruit, city;
    int quantity;
    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity; }
    public String toString() {
        return String.format("%-15s %-10s %-15s %d", name, fruit, city, quantity); }
}

public class JavaProgram {
    static void saveToFile(ArrayList<Farmer> list, String filename) throws IOException {
        BufferedWriter bw = new BufferedWriter(new FileWriter(filename));
        bw.write(String.format("%-15s %-10s %-15s %s%n",
                "Name", "Fruit", "City", "Quantity"));
        bw.write("-------------------------------------------------\n");
        for (Farmer f : list) {
            bw.write(f.toString() + "\n"); }
        bw.close();
        System.out.println("Saved: " + filename); }

    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        ArrayList<Farmer> farmers = new ArrayList<>();
        System.out.println("Enter details for 10 farmers:");
        for (int i = 1; i <= 10; i++) {
            System.out.println("Farmer " + i + ":");
            System.out.print(" Name : "); String name = sc.next();
            System.out.print(" Fruit : "); String fruit = sc.next();
            System.out.print(" City : "); String city = sc.next();
            System.out.print(" Quantity : "); int qty = sc.nextInt();
            farmers.add(new Farmer(name, fruit, city, qty)); }
        saveToFile(farmers, "before_sort.txt");

        Collections.sort(farmers, new Comparator<Farmer>() {
            public int compare(Farmer a, Farmer b) {
                return a.quantity - b.quantity; } });
        saveToFile(farmers, "after_sort.txt");
        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.next();
        boolean found = false;
```

```
        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("Found: " + f);
                found = true; } }
        if (!found) System.out.println("Farmer not found.");
        sc.close(); }
}
```

OUTPUT:

```
Enter details for 10 farmers:
Farmer 1:

 Name : ramesh
 Fruit : mango
 City : bhopal
 Quantity : 1200

Farmer 2:

 Name : suresh
 Fruit : apple
 City : indore
 Quantity : 450

Farmer 3:

 Name : mahesh
 Fruit : guava
 City : jabalpur
 Quantity : 800

Farmer 4:

 Name : rajnikant
 Fruit : peache
 City : ambikapur
 Quantity : 360

Farmer 5:

 Name : raju
 Fruit : berry
 City : rewa
 Quantity : 550

Farmer 6:

 Name : ganesh
 Fruit : papaya
 City : banglore
 Quantity : 225

Farmer 7:

 Name : sonu
 Fruit : banana
 City : mumbai
 Quantity : 300

Farmer 8:

 Name : roshan
 Fruit : pineapple
 City : jaipur
 Quantity : 448

Farmer 9:

 Name : saroj
 Fruit : kiwi
 City : raipur
 Quantity : 764

Farmer 10:

 Name : bhaskar
 Fruit : grape
 City : sambalpur
 Quantity : 540

Saved: before_sort.txt
Saved: after_sort.txt

Enter farmer name to search: suresh
Found: suresh      apple     indore      450
```

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

Ans:

```java
import java.util.*;

// enum for quality - only 3 fixed values allowed
enum Quality { GOOD, BAD, AVERAGE }

// Fruit class to store name, quantity, quality
class Fruit {
    String name;
    int quantity;
    Quality quality;
    Fruit(String name, int quantity, Quality quality) {
        this.name = name;
        // if quantity > 1000, store half
        this.quantity = (quantity > 1000) ? quantity / 2 : quantity;
        this.quality = quality; }
    public String toString() {
        return name + " | Qty: " + quantity + " | Quality: " + quality; }
}

public class JavaProgram {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] fruitNames = {"Mango", "Apple", "Papaya"};
        ArrayList<Fruit> list = new ArrayList<>();

        // Take input for each fruit
        for (String fname : fruitNames) {
            System.out.print("Enter quantity of " + fname + ": ");
            int qty = sc.nextInt();
            System.out.print("Enter quality (GOOD / BAD / AVERAGE): ");
            Quality ql = Quality.valueOf(sc.next().toUpperCase());
            list.add(new Fruit(fname, qty, ql)); }

        // Search by fruit name AND quality
        System.out.print("\nSearch - Enter fruit name: ");
        String sName = sc.next();
        System.out.print("Search - Enter quality (GOOD / BAD / AVERAGE): ");
        Quality sQl = Quality.valueOf(sc.next().toUpperCase());
        boolean found = false;
        for (Fruit f : list) {
            // both conditions must match
            if (f.name.equalsIgnoreCase(sName) && f.quality == sQl) {
                System.out.println("Found: " + f);
                found = true; } }
        if (!found) System.out.println("No match found."); }
}
```

OUTPUT:

```
Enter quantity of Mango: 1500
Enter quality (GOOD / BAD / AVERAGE): good

Enter quantity of Apple: 400
Enter quality (GOOD / BAD / AVERAGE): bad

Enter quantity of Papaya: 800
Enter quality (GOOD / BAD / AVERAGE): average

Search - Enter fruit name: papaya
Search - Enter quality (GOOD / BAD / AVERAGE): average

Found: Papaya | Qty: 800 | Quality: AVERAGE
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively. The available orchards in a tree-view. Search for a given fruit in the available orchards after taking input from the user & print the tree.

Ans:

```java
import java.util.*;

// Node Class – Represents an orchard or a fruit in the tree.
class Node {
    String name;
    List<Node> children = new ArrayList<>();
    Node(String name) {
        this.name = name; }
    void addChild(Node child) {
        children.add(child); }
}

public class JavaProgram {
    // Prints the Tree—the indentation reveals the level.
    static void printTree(Node node, String indent) {
        System.out.println(indent + "+-- " + node.name);
        for (Node child : node.children) {
            printTree(child, indent + "    "); } }

    // Searches for fruit from DFS — also prints the path.
    static boolean search(Node node, String target, String path) {
        if (node.name.toLowerCase().contains(target.toLowerCase())) {
            System.out.println("Found in: " + path);
            return true; }
        boolean found = false;
        for (Node child : node.children) {
            if (search(child, target, path + " -> " + child.name)) {
                found = true; } }
```

```java
        return found; }
    public static void main(String[] args) {
        // Root node
        Node root = new Node("Orchards");
        // Orchard A - Apple (Good)
        Node a = new Node("Orchard A");
        a.addChild(new Node("Apple [Good]"));
        // Orchard B - Litchi (Good)
        Node b = new Node("Orchard B");
        b.addChild(new Node("Litchi [Good]"));
        // Orchard C - Mango (Good)
        Node c = new Node("Orchard C");
        c.addChild(new Node("Mango [Good]"));
        // Orchard D - Apple (Bad)
        Node d = new Node("Orchard D");
        d.addChild(new Node("Apple [Bad]"));

        root.addChild(a);
        root.addChild(b);
        root.addChild(c);
        root.addChild(d);

        // Print tree view
        System.out.println("=== Orchard Tree ===");
        printTree(root, "");
        // Search
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter fruit to search: ");
        String target = sc.nextLine();
        System.out.println("\nSearch Results:");
        if (!search(root, target, "Orchards")) {
            System.out.println("Fruit not found in any orchard."); }
    }
}
```

OUTPUT:

```
=== Orchard Tree ===
+-- Orchards
  +-- Orchard A
    +-- Apple [Good]
  +-- Orchard B
    +-- Litchi [Good]
  +-- Orchard C
    +-- Mango [Good]
  +-- Orchard D
    +-- Apple [Bad]

Enter fruit to search: apple
Search Results:
Found in: Orchards -> Orchard A -> Apple [Good]
Found in: Orchards -> Orchard D -> Apple [Bad]
```

4. Take a map provided as a graph & display all the subtrees of connected points.

Ans:

```java
import java.util.*;

public class JavaProgram {
    // Graph stored as adjacency list
    // each node maps to a list of its connected nodes
    static Map<Integer, List<Integer>> graph = new HashMap<>();
    static void addEdge(int u, int v) {
        // undirected graph - add both sides
        graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u); }
    // DFS - visits all reachable nodes from 'node'
    // adds each visited node to subtree list
    static void dfs(int node, boolean[] visited, List<Integer> subtree) {
        visited[node] = true;
        subtree.add(node);
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, subtree); } } }

    public static void main(String[] args) {
        // Build the graph with edges
        addEdge(1, 2);
        addEdge(1, 3);
        addEdge(2, 4);
        addEdge(3, 5);
        addEdge(5, 6);
          /*
         Graph looks like this:
              1
             / \
            2   3
            |   |
            4   5
                |
                6
        */
        int totalNodes = 6;
        System.out.println("Subtrees from each node:");
        System.out.println("------------------------");
        // For each node, run DFS and collect all reachable nodes
        for (int start = 1; start <= totalNodes; start++) {
            boolean[] visited = new boolean[totalNodes + 1];
            List<Integer> subtree = new ArrayList<>();

            dfs(start, visited, subtree);

            System.out.println("From Node " + start + " : " + subtree); } }
}
```

OUTPUT:

```
Subtrees from each node:
------------------------
From Node 1 : [1, 2, 4, 3, 5, 6]
From Node 2 : [2, 1, 3, 5, 6, 4]
From Node 3 : [3, 1, 2, 4, 5, 6]
From Node 4 : [4, 2, 1, 3, 5, 6]
From Node 5 : [5, 3, 1, 2, 4, 6]
From Node 6 : [6, 5, 3, 1, 2, 4]
```

5. Enter entry point & goal point & produce the matching subtree of the graph.

Ans:

```java
import java.util.*;

public class JavaProgram {
    // Same adjacency list graph
    static Map<Integer, List<Integer>> graph = new HashMap<>();
    // path list - stores nodes visited on current route
    static List<Integer> path = new ArrayList<>();
    static void addEdge(int u, int v) {
        graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u); }
    // DFS with backtracking to find path from current to goal
    static boolean dfs(int current, int goal, boolean[] visited) {
        visited[current] = true;
        path.add(current);  // add to path
        // goal reached - stop here
        if (current == goal) {
            return true; }
        for (int neighbor : graph.getOrDefault(current, new ArrayList<>())) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited)) {
                    return true;  /* goal found in this branch */ }
            }
        }
        // dead end reached - backtrack
        // remove current node from path and go back
        path.remove(path.size() - 1);
        return false; }
    public static void main(String[] args) {
        // Build same graph as Q4
        addEdge(1, 2);
        addEdge(1, 3);
        addEdge(2, 4);
        addEdge(3, 5);
        addEdge(5, 6);
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter entry point: ");
        int entry = sc.nextInt();
        System.out.print("Enter goal point : ");
        int goal = sc.nextInt();
        boolean[] visited = new boolean[10];
        if (dfs(entry, goal, visited)) {
            System.out.println("\nPath found     : " + path);
            System.out.println("Nodes explored : " + path.size());
        } else {
            System.out.println("No path found from " + entry + " to " + goal); }
    }
}
```

OUTPUT:

```
Enter entry point: 1
Enter goal point : 6
Path found    : [1, 3, 5, 6]
Nodes explored : 4

Enter entry point: 2
Enter goal point : 5
Path found    : [2, 1, 3, 5]
Nodes explored : 4

Enter entry point: 3
Enter goal point : 2
Path found    : [3, 1, 2]
Nodes explored : 3
```

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

Ans:

```
import java.util.*;

public class JavaProgram {
    static Map<Integer, List<int[]>> graph = new HashMap<>();
    static void addEdge(int u,int v,int w){
        graph.computeIfAbsent(u,k->new ArrayList<>()).add(new int[]{v,w});
        graph.computeIfAbsent(v,k->new ArrayList<>()).add(new int[]{u,w});
}
public static void main(String[] args){
    Scanner sc=new Scanner(System.in);
```

```java
    // input edges
    System.out.print("Enter number of edges: ");
    int e=sc.nextInt();
    for(int i=0;i<e;i++){
        System.out.print("Enter u v weight: ");
        int u=sc.nextInt();
        int v=sc.nextInt();
        int w=sc.nextInt();
        addEdge(u,v,w); }
    System.out.print("Enter start node: ");
    int start=sc.nextInt();
    System.out.print("Enter end node: ");
    int end=sc.nextInt();
    // dijkstra
    int[] dist=new int[100];
    Arrays.fill(dist,Integer.MAX_VALUE);
    dist[start]=0;
    PriorityQueue<int[]> pq=new PriorityQueue<>((a,b)->a[1]-b[1]);
    pq.add(new int[]{start,0});
    while(!pq.isEmpty()){
        int[] cur=pq.poll();
        for(int[] nei:graph.getOrDefault(cur[0],new ArrayList<>())){
            int next=nei[0],wt=nei[1];
            if(dist[next]>cur[1]+wt){
                dist[next]=cur[1]+wt;
                pq.add(new int[]{next,dist[next]}); } } }
                System.out.println("Shortest distance = "+dist[end]);
    }
    public static Map<Integer, List<int[]>> getGraph() {
        return graph; }
    public static void setGraph(Map<Integer, List<int[]>> graph) {
        JavaProgram.graph = graph; }
}
```

OUTPUT:

```
Enter number of edges: 4
Enter u v weight: 1 2 3
Enter u v weight: 1 3 2
Enter u v weight: 2 4 1
Enter u v weight: 3 4 5
Enter start node: 1
Enter end node: 4
Shortest distance = 4
```

7.Write a program to demonstrate the use of all keywords made for exception handling.

Ans:

```java
// Custom exception - extends Exception (checked)
class MyException extends Exception {
    MyException(String msg) {
        super(msg); // pass message to parent Exception class
    } }
public class JavaProgram {

    // 'throws' keyword - tells caller this method may throw MyException
    static void checkAge(int age) throws MyException {
        if (age < 0) {
            // 'throw' keyword - manually triggers the exception
            throw new MyException("Age cannot be negative: " + age); }
        System.out.println("Valid age: " + age); }
    public static void main(String[] args) {

        // Test 1: Built-in ArithmeticException
        System.out.println("--- Test 1: Divide by zero ---");
        try {
            int result = 10 / 0; // this line throws ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        } finally {
            // finally always runs - used for cleanup
            System.out.println("Finally block ran."); }

        // Test 2: Custom exception using throw and throws
        System.out.println("\n--- Test 2: Custom Exception ---");
        try {
            checkAge(-5); // will throw MyException
        } catch (MyException e) {
            System.out.println("Caught MyException: " + e.getMessage());
        } finally {
            System.out.println("Finally block ran."); }

        // Test 3: Multiple catch blocks
        System.out.println("\n--- Test 3: Null Pointer ---");
        try {
            String s = null;
            s.length(); // throws NullPointerException
        } catch (NullPointerException e) {
            System.out.println("Caught: NullPointerException");
        } catch (Exception e) {
            // catches any other exception
            System.out.println("Caught general: " + e.getMessage());
        } finally {
            System.out.println("Finally block ran."); }
        System.out.println("\nProgram ends normally.");
    }
}
```

OUTPUT:

```
--- Test 1: Divide by zero ---
Caught: / by zero
Finally block ran.

--- Test 2: Custom Exception ---
Caught MyException: Age cannot be negative: -5
Finally block ran.

--- Test 3: Null Pointer ---
Caught: NullPointerException
Finally block ran.
```

8. Write a program to withdraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

Ans:

```java
import java.util.Scanner;

// Custom exception for insufficient balance
class NoMoneyException extends Exception {
    NoMoneyException(String msg) {
        super(msg); } }
class BankAccount {
    String owner;
    double balance;
    BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance; }

    // 'throws' declares that this method can throw NoMoneyException
    void withdraw(double amount) throws NoMoneyException {
        if (amount > balance) {
            // throw triggers the exception with a message
            throw new NoMoneyException(
                "Cannot withdraw Rs." + amount +
                "  Available balance: Rs." + balance ); }
        balance -= amount;
        System.out.println("Withdrawal successful : Rs." + amount);
        System.out.println("Remaining balance     : Rs." + balance); } }

public class JavaProgram {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Developer", 5000);
        System.out.println("Account : " + acc.owner);
        System.out.println("Balance : Rs." + acc.balance);
```

```java
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter amount to withdraw: Rs.");
        double amount = sc.nextDouble();
        try {
            acc.withdraw(amount);
        } catch (NoMoneyException e) {
            // getMessage() prints the message we wrote in throw
            System.out.println("ERROR: " + e.getMessage());
        } finally {
            System.out.println("Transaction attempt complete."); } }
}
```

OUTPUT:

1st:

Account : Developer
Balance : Rs.5000.0

Enter amount to withdraw: Rs.5001
ERROR: Cannot withdraw Rs.5001.0
Available balance: Rs.5000.0
Transaction attempt complete.

2nd:

Account : Developer
Balance : Rs.5000.0

Enter amount to withdraw: Rs.4000
Withdrawal successful : Rs.4000.0
Remaining balance : Rs.1000.0
Transaction attempt complete.

9. Write a program which enables to use a platform by a train one at a time.

Ans:

```java
class Platform {
    boolean occupied = false; // true = platform is busy
    // synchronized - only one thread can enter this method at a time
    synchronized void usePlatform(String trainName) {
        // if platform is busy, this train waits
        while (occupied) {
            try {
                System.out.println(trainName + " is waiting...");
                wait(); // releases lock and pauses this thread
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); } }

        // platform is free - train enters
        occupied = true;
        System.out.println(trainName + " arrived at platform.");
        try {
            Thread.sleep(1500); // simulate time spent at platform
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt(); }
        System.out.println(trainName + " left the platform.");
        occupied = false;
        notifyAll(); // wake all waiting trains
    } }

// Each train is a separate thread
class Train implements Runnable {
    String name;
    Platform platform;
    Train(String name, Platform platform) {
        this.name = name;
        this.platform = platform; }
    public void run() {
        platform.usePlatform(name); // try to use the platform
    } }

public class JavaProgram {
    public static void main(String[] args) {
        Platform platform = new Platform();

        // Create 3 trains as separate threads
        Thread t1 = new Thread(new Train("Express 101", platform));
        Thread t2 = new Thread(new Train("Rajdhani 202", platform));
        Thread t3 = new Thread(new Train("Shatabdi 303", platform));
        t1.start();
        t2.start();
        t3.start(); }
}
```

OUTPUT:

```
Express 101 arrived at platform.

Express 101 left the platform.

Shatabdi 303 arrived at platform.

Shatabdi 303 left the platform.

Rajdhani 202 arrived at platform.

Rajdhani 202 left the platform.
```

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and chef deliver orders if he is free.

Ans:

```java
class Kitchen {
    String currentOrder = null;
    // null means no order is pending
    // non-null means an order is waiting to be delivered
    // Waiter calls this to place a new order
    synchronized void placeOrder(String item) {
        // if an order is already pending, waiter waits
        while (currentOrder != null) {
            try {
                System.out.println("[Waiter] Kitchen busy, waiting...");
                wait(); // releases lock, pauses waiter thread
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); } }

        // kitchen is free - place the order
        currentOrder = item;
        System.out.println("[Waiter] Order placed : " + item);
        notify(); // wake the chef
    }
    // Chef calls this to pick up and deliver order
    synchronized void deliverOrder() {
        // if no order is ready, chef waits
        while (currentOrder == null) {
            try {
                System.out.println("[Chef]   No order yet, waiting...");
                wait(); // releases lock, pauses chef thread
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); } }
        // order is ready - deliver it
        System.out.println("[Chef]   Delivering  : " + currentOrder);
        currentOrder = null; // kitchen is free now
        notify(); // wake the waiter for next order
    } }
```

```java
// Waiter thread - places all orders one by one
class Waiter implements Runnable {
    Kitchen kitchen;
    Waiter(Kitchen kitchen) {
        this.kitchen = kitchen; }
    public void run() {
        String[] orders = {"Burger", "Pizza", "Pasta", "Salad", "Soup"};
        for (String order : orders) {
            kitchen.placeOrder(order); } } }
// Chef thread - delivers all orders one by one
class Chef implements Runnable {
    Kitchen kitchen;
    int totalOrders;
    Chef(Kitchen kitchen, int totalOrders) {
        this.kitchen = kitchen;
        this.totalOrders = totalOrders; }
    public void run() {
        for (int i = 0; i < totalOrders; i++) {
            kitchen.deliverOrder(); } } }

public class JavaProgram {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();
        int totalOrders = 5;
        Thread waiterThread = new Thread(new Waiter(kitchen));
        Thread chefThread   = new Thread(new Chef(kitchen, totalOrders));
        System.out.println("=== Restaurant Open ===\n");
        // start both threads
        chefThread.start();
        waiterThread.start(); } }
```

OUTPUT:

=== Restaurant Open === [Chef] No order yet, waiting... [Waiter] Order placed : Burger [Waiter] Kitchen busy, waiting... [Chef] Delivering : Burger [Chef] No order yet, waiting... [Waiter] Order placed : Pizza [Waiter] Kitchen busy, waiting... [Chef] Delivering : Pizza [Chef] No order yet, waiting... [Waiter] Order placed : Pasta [Waiter] Kitchen busy, waiting... [Chef] Delivering : Pasta	[Chef] No order yet, waiting... [Waiter] Order placed : Salad [Waiter] Kitchen busy, waiting... [Chef] Delivering : Salad [Chef] No order yet, waiting... [Waiter] Order placed : Soup [Chef] Delivering : Soup